Local C++ setup in VSCode (Windows / macOS / Linux)

If VSCode is already installed and ready to use, install g++, then check its version and compile and run Hello World in the VSCode integrated terminal to complete setup. Use the C++17 standard.

If VSCode is not configured, follow the existing CLion setup: CLion Chinese guide · CLion English guide.

1. Install g++ for your operating system

This guide is for students who have already installed VSCode and can use it. Keep your existing settings and install the compiler for your operating system. **Installing VSCode or the C/C++ extension does not install g++**. If `g++ -v` already works in the VSCode terminal, go directly to the Hello World check in Section 2.

Select **Terminal → New Terminal**. Use the Terminal panel that opens below to check the compiler, compile your code, and run it. Do not enter these commands in Output or Debug Console. Use PowerShell on Windows, or zsh or bash on macOS/Linux.

Windows

Reference video: C++ environment setup on Windows. If VSCode is already installed, focus on installing the compiler and configuring PATH, then return to this guide for verification.

The following steps install g++ through MSYS2 on Windows x64:

1. Download and install MSYS2 from the MSYS2 website. These steps assume the default installation directory, `C:\msys64`.

2. Open **MSYS2 UCRT64** from the Start menu and run:

```
pacman -S --needed mingw-w64-ucrt-x86_64-gcc
```

Press Enter or type `Y` when asked to confirm, and wait for installation to finish. Run this installation command in MSYS2 UCRT64.

3. Search Windows for "Edit environment variables for your account". Edit the user variable `Path` and add `C:\msys64\ucrt64\bin`. If you installed MSYS2 elsewhere, use the corresponding `ucrt64\bin` directory.

4. Save the environment variables, close all VSCode windows completely, and reopen VSCode. Select **Terminal → New Terminal**, use PowerShell, and run:

```
g++ -v
```

When GCC version and compiler information appear, continue to Hello World below. If g++ is not found, check that PATH points to the directory containing `g++.exe` and that you have restarted VSCode.

macOS: install Xcode Command Line Tools

On macOS, use the compiler provided by Xcode Command Line Tools. Run this command in the VSCode integrated terminal:

```
xcode-select --install
```

Follow the installation dialog and wait for installation to finish. If the tools are already installed, continue to the next step. You do not need the full Xcode application.

Open a new terminal in VSCode and run:

```
g++ -v
```

Apple Clang version information is the expected result: on macOS, the g++ command invokes Apple Clang. It can compile the course programs with `g++ -std=c++17` as shown in this guide; installing GNU GCC separately is unnecessary.

Linux: check first and install if needed

Some Linux environments already have g++, but it is not included in every distribution by default. First run this command in the VSCode integrated terminal:

```
g++ -v
```

If compiler version information appears, go directly to Section 2. If the command is not found, choose the installation commands for your distribution.

Ubuntu / Debian:

```
sudo apt update
sudo apt install build-essential
```

Fedora:

```
sudo dnf install gcc-c++
```

Wait for installation to finish, then run `g++ -v` again in the VSCode terminal and confirm that GCC version information appears.

2. Compile and run Hello World in the VSCode terminal

After installing the compiler, complete the checks in this section. Run all version checks, compilation commands, and execution commands in the **VSCode integrated terminal**. This section uses manual compilation and execution; it does not require `tasks.json`, `launch.json`, or Code Runner.

1. Open a practice folder and check the compiler

Create a local practice folder, such as `cpp-hello`, open it in VSCode through **File → Open Folder**, open a new terminal, and run:

```
g++ -v
```

Confirm that compiler version information appears and that there is no "command not found" error.

2. Create and save hello.cpp

Create `hello.cpp` in the folder you opened, enter the following code, and save it. You can also use the accompanying hello.cpp:

```
#include <iostream>

int main() {
    std::cout << "Hello World\n";
    return 0;
}
```

Make sure the filename is `hello.cpp`, not `hello.cpp.txt`, and that the terminal's current directory is the folder containing this file. Save any changes before compiling.

3. Compile and run

Windows (PowerShell): Run these two lines separately. Run the second line only after the first succeeds.

```
g++ -std=c++17 hello.cpp -o hello.exe
.\hello.exe
```

macOS / Linux: Run these two lines separately. Run the second line only after the first succeeds.

```
g++ -std=c++17 hello.cpp -o hello
./hello
```

Successful compilation usually produces no output. Running the program should display the following in the same terminal:

```
Hello World
```

If the compiler cannot find `hello.cpp`, check your current directory with `Get-Location` and `dir` on Windows, or `pwd` and `ls` on macOS/Linux. Change to the folder containing the source file and compile again.

4. Completion checklist

☐ Run `g++ -v` in the VSCode integrated terminal and see compiler version information.

☐ Successfully compile `hello.cpp` with `g++ -std=c++17` in the same terminal.

☐ Run the generated program and see `Hello World` in the terminal.